

Reviewer Pack — Matroid Rank Inverse Proportional to Monotone DNF Discrepanc...

Entry 70db22789577 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 12:05:43 UTC

Matroid Rank Inverse Proportional to Monotone DNF Discrepancy for k-CLIQUE

Entry ID: 70db22789577 Verdict: INCONCLUSIVE Recorded: 2026-05-10 12:05:43 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone DNF Discrepancy

Statement:

For a k-CLIQUE instance, the discrepancy of its monotone DNF representation is $\Theta(1/\text{rank}(M))$, where M is the vector matroid over $\text{GF}(2)$ derived from the incidence matrix of the formula’s clauses.

Rationale (proposer’s reasoning):

Matroid rank captures the combinatorial structure of clause dependencies, while discrepancy quantifies the uniformity of clause coverage. Their inverse proportionality suggests inherent complexity barriers for monotone circuits.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1b9077a5621e6e8c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(M):
    n = len(M)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]

        # Eliminate non-pivot elements
        for j in range(n):
            if i != j:
                factor = Fraction(M[j][i], M[i][i])
                for k in range(i, n+1):
                    M[j][k] -= factor * M[i][k]

    rank = 0
    for row in M:
        if any(row):
            rank += 1
    return rank

def discrepancy(clauses, variables):
    max_discrepancy = 0
    for S in range(1 << len(variables)):
        satisfied_clauses = sum(all(v in S for v in clause) for clause in clauses)
        max_discrepancy = max(max_discrepancy, abs(satisfied_clauses - len(clauses)))
    return max_discrepancy

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    k = min(n // 2, 10)
    clauses = []
```

```

for _ in range(k):
    clause = set(random.sample(range(n), random.randint(1, n)))
    clauses.append(clause)

incidence_matrix = [[int(v in clause) for v in range(n)] for clause in clauses]

rank = gaussian_elimination(incidence_matrix)
discrepancy_value = discrepancy(clauses, range(n))

expected_discrepancy = Fraction(1, rank)
if discrepancy_value != expected_discrepancy:
    return {
        "metric_name": "discrepancy",
        "metric_value": discrepancy_value,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Discrepancy {discrepancy_value} does not match expected {expected_discrepancy}"
    }

    return {
        "metric_name": "discrepancy",
        "metric_value": discrepancy_value,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_discrepancy = sum(r["metric_value"] for r in results) / len(results)
    std_discrepancy = math.sqrt(sum((r["metric_value"] - mean_discrepancy)**2 for r in results) / len(results))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_discrepancy} std={std_discrepancy} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5045b20c.py", line 86, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5045b20c.py", line 60, in run_trial
    rank = gaussian_elimination(incidence_matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5045b20c.py", line 31, in gaussian_elim
    factor = Fraction(M[j][i], M[i][i])
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError during Gaussian elimination, preventing result validation. | next: Inspect incidence matrix construction to resolve division-by-zero error in Gaussian elimination

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82760
2	preregistration	ollama_remote	qwen3:8b	0	0	24121
3	novelty	ollama_remote	qwen3:8b	0	0	21223
4	novelty	ollama_remote	qwen3:8b	0	0	17783
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14624
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9749
7	judge	ollama_remote	qwen3:8b	0	0	18708

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 188967 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/70db22789577.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/70db22789577.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/70db22789577.tar.gz` (if generated)